

OS Practical No. 10

Aim: Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The SSTF scheduling algorithm is used. The head is initially at cylinder number 53. The cylinders are numbered from 0 to 199. Write a Program to calculate the total head movement (in number of cylinders) incurred while servicing these requests.

Code :

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int request[] = {98, 183, 41, 122, 14, 124, 65, 67};
    int visited[8] = {0};
    int head = 53;
    int total_movement = 0;
    int n = 8;

    printf("Order of servicing:\n");

    for(int i = 0; i < n; i++)
    {
        int min = 9999;
        int index = -1;

        for(int j = 0; j < n; j++)
        {
            if(!visited[j])
            {
                int distance = abs(head - request[j]);
                if(distance < min)
                {
                    min = distance;
                    index = j;
                }
            }
        }

        visited[index] = 1;
```

```

        total_movement += abs(head - request[index]);
        head = request[index];

        printf("%d ", head);
    }
    printf("\nTotal Head Movement = %d\n", total_movement);

    return 0;
}

```

Output :

```

Last login: Sun Mar 15 13:03:30 on console
[saumyaudgirkar@Mac ~ % git --version
git version 2.52.0
[saumyaudgirkar@Mac ~ % nano sstf.c
[saumyaudgirkar@Mac ~ % gcc sstf.c -o sstf
[saumyaudgirkar@Mac ~ % ./sstf
Order of servicing:
41 65 67 98 122 124 183 14
Total Head Movement = 323
saumyaudgirkar@Mac ~ % █

```

```

UW PICO 5.09

#include <stdio.h>
#include <stdlib.h>

int main()
{
    int request[] = {98, 183, 41, 122, 14, 124, 65, 67};
    int visited[8] = {0};
    int head = 53;
    int total_movement = 0;
    int n = 8;

    printf("Order of servicing:\n");

    for(int i = 0; i < n; i++)
    {
        int min = 9999;
        int index = -1;

        for(int j = 0; j < n; j++)
        {
            if(!visited[j])
            {
                int distance = abs(head - request[j]);
                if(distance < min)
                {
                    min = distance;
                    index = j;
                }
            }
        }

        visited[index] = 1;
        total_movement += abs(head - request[index]);
        head = request[index];

        printf("%d ", head);
    }

    printf("\nTotal Head Movement = %d\n", total_movement);

    return 0;
}

```